CpSc 418

Homework 1

Getting started with Erlang

Due: September 16, 2025, 11:59pm
Early-Bird: September 14, 2025, 11:59pm

Prelude

Please submit your solution using:
`handin cs-418 hw1` Your solution should contain two files:

`hw1.erl`: Erlang source code for your solutions to the questions.

`hw1.pdf`: Your solutions to written questions.

The following files are available at http://www.students.cs.ubc.ca/~cs-418/2024-1/hw/1/hw1.html:

`hw1.erl`: a template for `hw1.erl`.

`hw1_tests.erl`: a few EUnit tests for `hw1.erl`.

`hw1x.beam`: a beam code for my solution.

Please submit code that compiles without errors or warnings. If your code does not compile, we might give you zero points on all of the programming problems. If we fix your code to make it compile, we will take off lots of points for that service. If your code generates compiler warnings, we will take off points for that as well, but not as many as for code that doesn't compile successfully. Any question about whether or not your code compiled as submitted will be determined by trying it on CS department linux machines.

We will take off points for code that prints results unless we specifically asked for print-out. For this assignment, the functions you write should return the specified values, but they should not print anything to `stdout`. Using lists:format when debugging is great, but you need to delete or comment-out such calls before submitting your solution. Your code must fail with some kind of error when called with invaid arguments.

This assignment is based on the first thirteen sections of *Learn You Some Erlang* – up through More on Multiprocessing.

1 The Questions (93 points)

0. **Collaborators** (**3 points**).
In your `hw1.pdf` list anyone you collaborated with or outside materials that you used.

   - If the collaborator is a student in this class, give their CWL.
   - If the colloborator is a human who is not in this class, give their name and a short (i.e. one sentence) description of their relevance to this course.
   - If you used on-line material, give the URL.
   - If you used an AI assistant, give the name of the assistant. We'll give extra credit if you include your prompt, and one sentence to say what was useful in the response.
   - Other sources such as books should be just cited clearly.

   For each collaborator, list what questions you collaborated on.

   You do not need to list course materials including anything on the cs-418 website, assigned readings, the official Erlang documentation, or material from office hours or tutorials.

   If you had no collaborators, write "*No collaborators*". A blank or omitted response will get no credit.
   No collaborators

Table 1: Patterns and Values

Pattern	Value
i. `X`	a. `[]`
ii. `_`	b. `0`
iii. `[X, Y]`	c. `[cat]`
iv. `[X | Y]`	d. `[cat, fish]`
v. `[X, Y | Z]`	e. `[potoroo, bettong, wombat ]`
vi. `0`	f. `lists:seq(10,0,-2)`
vii. `{X}`	g. `{bat, wombat}`
viii. `{X,_}`	h. `[{panda, 3}, {penguin, 2}, {potoroo, 137}]`
ix. `[_, {_,X} | _]`	i. `[{dog, 2}, {potoroo, 137}, {cat, 3}]`
x. `[_|_]`	j. `{[cat, dog, potoroo], [3, 2, 137]}`

1. **Patterns** (**10 points**).
For each `Pattern` in Table 1 list the `Values` from the table that match it. If `Pattern` includes the variable `X`, then for each `Value` that matches it, give the value of `X`.

Pattern	Matching Values
i. `X`	a. `X=[]`; b. `X=0`; c. `X=[cat]`; d. `X=[cat,fish]`; e. `X=[potoroo,bettong,wombat]`; f. `X=[10,8,6,4,2,0]`; g. `X={bat,wombat}`; h. `X=[{panda,3},{penguin,2},{potoroo,137}]`; i. `X=[{dog,2},{potoroo,137},{cat,3}]`; j. `X={[cat,dog,potoroo],[3,2,137]}`
ii. `_`	a, b, c, d, e, f, g, h, i, j
iii. `[X, Y]`	d. `X=cat`
iv. `[X | Y]`	c. `X=cat`; d. `X=cat`; e. `X=potoroo`; f. `X=10`; h. `X={panda,3}`; i. `X={dog,2}`
v. `[X, Y | Z]`	d. `X=cat`; e. `X=potoroo`; f. `X=10`; h. `X={panda,3}`; i. `X={dog,2}`
vi. `0`	b
vii. `{X}`	(no matches)
viii. `{X,_}`	g. `X=bat`; j. `X=[cat,dog,potoroo]`
ix. `[_, {_,X} | _]`	h. `X=2`; i. `X=137`
x. `[_|_]`	c, d, e, f, h, i

2. **Modular exponentiation.** (15 points)
We want to compute X^N mod `M` where `X`, `N`, and `M` are all integers. An obvious implementation is:

```
mpow_0(X, N, M) when is_integer(X), is_integer(N), 0 =< N, is_integer(M), 0 < M ->
  misc:mod(X*mpow_0(X, N-1, M), M);
mpow_0(X, 0, 1) when is_integer(X) -> 0;
mpow_0(X, 0, M) when is_integer(X), is_integer(M), 1 < M -> 1.
```

Why the `_0` subscript? Because we're going to write a better version below, and I'm saving the name `mpow` for the better version. Why the guards of `is_integer(X)` for all three cases? Because I wrote above that the Erlang functions we write should throw an exception when called with invalid arguments.

(a) Russian Peasant exponentiation (5 points)
The Russian Peasant method is defined by the following recurrence:

$$\begin{aligned} X^0 &= 1 \\ X^n &= (X^2)^{n/2}, && \text{if } n \text{ is even} \\ X^n &= X * (X^2)^{(n-1)/2}, && \text{if } n \text{ is odd} \end{aligned}$$

Write `mpow(X, N, M)` using the Russian Peasant algorithm. Make sure that you take the intermediate results modulo-`M` at each step; otherwise you can produce *enormous* intermediate results which will make your code run *really* slowly. In Erlang, `N/2` produces a floating point value even if `N` is an even integer. Use `div`, truncating integer division when you need an integer result.

(b) Is the Russian Peasant algorithm fast? (10 points)

i. **(5 points)**
Use `time_it:t` from the course library to compare the execution times for `mpow_0(X, N, M)` and `mpow(X, N, M)` for `X=1234567`, `M=10000000`, and `N` $\in$ {`1000, 10000, 100000, 500000`}. Please measure all times on `thetis.students.cs.ubc.ca` or `thetis.students.cs.ubc.ca`, and report the times in a table.

Execution time comparison between `mpow_0` and `mpow`

N	mpow_0 (seconds)	mpow (seconds)
1,000	4.35×10^{-5}	8.94×10^{-7}
10,000	5.05×10^{-4}	7.59×10^{-7}
100,000	4.01×10^{-3}	1.11×10^{-6}
500,000	2.16×10^{-2}	1.14×10^{-6}

ii. **(5 points)** Let

```
Ipow = fun F(_,0) -> 1;
       F(X,N) when is_integer(N), N > 0 -> X*F(X,N-1)
       end,
Googol = Ipow(10, 100),
BigInteger = lists:foldl(fun(X, Prod) -> X*Prod end, 1, lists:seq(123456789, 123456800)).
```

Use `time_it:t` from the course library to measure the execution time for

```
hw1:mpow(BigInteger+17, BigInteger, Googol)
```

- What time did you measure?
  Note: you can try this with `hw1x:mpow` to see if your implementation is "reasonable".
  `8.503` $\times$ `10`^{-4}s
- Why would you not want to use `mpow_0` for this computation?
  A one sentence answer is sufficient.
  Using mpow_0 on a large number (or any number) will result in N function calls, while mpow will result in $log_2 N$ function calls.

3. **Rabin-Miller primality testing** (30 points):
The Rabin-Miller algorithm is a cool algorithm, it is widely used (e.g. for generating RSA keys), and I'll use it to create "embarrassingly parallel" problem for Question 5. This problem comes with a two-page introduction – I've tried to keep the number theory informal and simple, but give hopefully give you enough understanding of the algorithm to appreciate it help you when debugging your implementation.

Motivation: Sometimes, we want to find some fairly large prime numbers. For example, the RSA-2048 encryption uses a public key that is the product of two prime numbers, where the product should be a 2048 bit integer. Thus, each the primes should be roughly 1024 bit integers, or around 10^{308}. How do we find primes with 300 or more digits?

The good news is that for numbers near a large integer, N, the fraction of such integers that are prime is roughly $1/\log(N)$, where log is the natural logarithm. See the Prime Number Theorem if you want the details. To find a random prime around 10^J we just need, on average, to try $\log(10^J) \approx 2.3J$

numbers. Because we don't need to test even numbers, we can reduce that to half as many, i.e. $1.15J$, and with further "tricks", we could reduce the constant factor even more.

Given a large, odd number, Q, how can we determine if it's prime? Testing all factors up to Q is clearly impractical. We only need to test factors up to $\sqrt{Q}$, but if we are looking for prime numbers around 10^{308} we still need to try 10^{154} possible factors for a brute force approach. We need something better, and for this problem, we'll use the Rabin-Miller algorithm. I'll sketch the ideas behind the algorithm here, and I'll try to keep the number-theory fairly informal.

The Rabin-Miller primality test is a randomized algorithm. There are three main ideas behind the algorithm:

Fermat's Little Theorem: *If P is a prime and A is an integer that is not a multiple of P, then* $A^{P-1} \bmod P = 1$.
See Appendix 2.2.1 for a proof and some examples.

The Square Root of 1: *If P is a prime and A is an integer such that $A * A \bmod P = 1$, then either* $A \bmod P = 1$ *or* $A \bmod P = P - 1$.
See Appendix 2.2.2 for a proof and some examples.

Randomized computation: Fermat's Little Theorem and the observation about squares mod P can be used to show that an alleged prime, Q is composite, but can't prove that Q is prime. The breakthrough that Rabin and Miller made is they combined these two in a way such that the probability that a composite number Q passes both tests for a randomly chosen A is at most $\frac{1}{4}$. If we choose K different values of A independently, then the probability of Q passing all K trials is at most 4^{-K}.

For example, if `K=50`, then the probability of incorrectly declaring `Q` to be prime is at most $4^{-50} \approx 7.9 \times 10^{-31}$. In other words, if we found one alleged prime per nanosecond using the Rabin-Miller test, we'd expect to have a non-prime slip through about once every 40 trillion years – about 300 times the age of the universe. If that's not good enough for you, just choose K large enough to meet your high standards.

Figure 1 sketches the Rabin-Miller algorithm. You now know "everything" you need to implement the Rabin-Miller algorithm. The functions we will use are:

`rabin_miller(Q, K)`: return `true` if `Q` passes `K` rounds of the Rabin-Miller test and `false` otherwise.

`rabin_miller(Q, K)` calls `rm_decompose/1` and `rm_check/2`.

An implementation of `rabin_miller(Q, K)` is provided in the `hw1.erl` template file. It just checks that the arguments are "reasonable" and handles some trivial cases to avoid special cases in the code you will write for other functions.

An implementation of `rabin_miller(Q)` is provided in the `hw1.erl` template file. It just calls `rabin_miller(Q, 50)`.

`rm_decompose(N)`: returns `{D,S}` such that `D` is a positive, odd number, `S` is a non-negative integer, and `N == D*2^S`.

- `N` must be a positive integer.
- `rm_decompose(N)` is called by `rabin_miller/2`.

`rm_check(Q, D, S, K)`: Perform `K` trials of the Rabin-Miller test for `Q`.

- Return `true` iff all trials pass.
- `D` and `S` must satisfy `D*2^S = Q-1`.
- `rm_check(Q, D, S, K)` is called by `rabin_miller/2`.
- `rm_check(Q, D, S, K)` calls `rm_check(Q, B, S)` with `B = mpow(A, D, Q)`.

`rm_check(Q, D, S)`: Perform one trial of the Rabin-Miller test.

1. Let `Q` be the number whose primality we wish to test. `Q` must be a positive, odd integer. Let `K` be a positive integer.
2. Find `D` and `S` such that `D` is odd and `Q-1` $= D * 2^S$.
3. Repeat `K` times:
   a. Pick a random integer, `A` with `1 < A < Q-1`.
   b. Raise `A` to the D^{th} power, mod `Q`. Let B_0 `= mpow(A, D, Q)`.
   c. If B_0 `== 1`, then continue with the next of the `K` iterations.
   d. Otherwise, compute B_0^S `rem Q` by repeated squaring:
      i. For `I` in `1` to `S`, let B_I `= (`B_{I-1} `* `B_{I-1}`) rem Q`. Remark: $B_I = B_0^{(2^I)}$ B_I `==` $B_0^{2^I}$ `rem Q`.
      ii. If for any `I` in `1` to `S`, B_I `== 1` and B_{I-1} is neither `1` nor `Q-1`, then return false. `Q` has failed the square-root test and is not prime.
      iii. If B_S `/= 1` then return `false`. `Q` has failed the Fermat test and is not prime.
      iv. If both conditions hold, then `Q` has passed this round of the Rabin-Miller algorithm.
4. If `Q` passes all `K` rounds, then the probability that `Q` is composite is at most 4^{-K}, and Rabin-Miller returns `true`.
5. Otherwise (i.e. `Q` failed either the square-root test or the Fermat test on one of the `K` rounds), Rabin-Miller returns `false`.

Figure 1: Outline of the Rabin-Miller primality testing algorithm.

- This corresponds to steps 3.a through 3.d from Figure 1.
- Return `true` iff for the random choice of `A`, `Q` passes the square-root and Fermat tests.
- `rm_check(Q, D, S)` is called by `rm_check/4`.

The actual questions! (finally)

(a) Implement `rm_decompose(N)`. (5 points)
See the description above and in the `hw1.erl` template file.

(b) Implement `rm_check(Q, D, S)`. (10 points)
See the description above and in the `hw1.erl` template file.
Hint: my solution computes `B0 = mpow(A, D, Q)` and then calls a helper function for the `S` squaring rounds.

(c) Implement `rm_check(Q, D, S, K)`. (10 points)
See the description above and in the `hw1.erl` template file.

(d) Give a brief description of your design and coding choices when solving questions 3b and 3c.

3b: The initial function generates a random number (A) by using the `rand:uniform` function. Once the random number is generated a helper function is called to iterate through the repeated squaring checks. The helper function will utilize tail elimination (memory efficient) as it recurses and does not depend on intermediate values. The helper function performs the repeated squaring by computing $B_i = B_{i-1}^2 \bmod Q$ for each iteration and checks both the square-root test and Fermat test conditions at each step.

3c: I chose to use the list builder notation to generate K elements all which call the 3 parameter `rm_check()`. Once the list is generated I use the `lists:all` function to determine if all the tests passed (entire list evaluates to true - K rounds passed).

4. Finding Primes. (10 points)
Now that we have a primality test, let's find some primes. Complete the function `find_primes(Lo, Hi, N)` in the `hw1.erl` template file. This function should return the first (i.e. smallest) `N` primes in the interval `[Lo, Hi]` (inclusive). If the number of primes in `[Lo, Hi]` is less than `N`, return a list of all primes in the interval. There should be no duplicates, but we won't be picky about the order of the list. You should use the Rabin-Miller test to determine whether or not a candidate integer is prime. A simple sieve is more efficient for finding small primes, but we're using this to test your implementation of the Rabin-Miller algorithm.

   Note: my solution calls a tail-recursive helper function. You don't need to do that, but if you find it simpler and clearer to implement `find_primes/3` using one or more helper functions, please do so.

5. Parallel Primes. (25 points)
Yay – we're finally doing something parallel! Implement `par_primes(Lo, Hi, N, NW)` which is like `find_primes(Lo, Hi, N)` but computed with `NW` worker processes. `par_primes(Lo, Hi, N, NW)` should:

   - Divide `[Lo, Hi]` into `NW` disjoint intervals.
   - Spawn `NW` worker processes, each should be tasked with finding `N/NW` primes (rounded up or down to make the total `N`) and sending its list of primes back to the original process.
   - Receive the lists of primes from the spawned processes and concatenate these lists to return a list of `N` primes from the interval `[Lo, Hi]`.

   The worker processes should terminate after they complete their computation.

   You should explicitly spawn the processes, and write the expressions for sending and receiving messages. While this functionality is present in the `workers` and `wtree` modules in the class library, we're asking you do implement the spawning, sending, and receiving here to get some experience with that level of message passing. All processes that you spawn should terminate when you `par_primes` returns a result.

   (a) **15 points**) Implement `par_primes` as described above.

   (b) **5 points**) Describe the design and coding choices you made in implementing `par_primes`. This should explain *why* you did what you did so that another programmer can understand your implementation.

   I determined the chunk size (range of values for each process to find primes) by taking $(Hi - Lo) div NW$. I then determined number of primes for each process to find (Dist_N: N div NW).

   Using recursion I then generated a list of tuples that contain each processes find_primes parameters {Lo, Hi, N}. The last worker will be allocated the remaining range and final required prime count, as Dist_N and chunk size are calculated by integer division. The reason i decide to make a list of tuples was to make it clear that the final process can potentially have different parameters than the rest.

   These parameters are then distributed to each process along with the parent process PID. The child processes responsibilities are to perform find_primes with the tuple parameters and send back the resulting list to the parent process. After initiating the child processes the parent process waits until it receives NW messages (prime lists) which it then concatenates for a final result.

   I beleive the way I implemented this would be quite clear to another programmer. The function determines what each processes parameters will be. It then distributes them to the child processes. Then it waits for the child processes results and concatenates them once they are received.

(c) **(3 points)** How many primes can you find in roughly one second without parallelism (i.e. just calling `find_primes`? Or with the parallel version using 4, 16, 32, 64, 128, and 256 processes? Make your measurements with `Lo = 3*ipow(2, 126)` and `Hi = 5*ipow(2, 126)` and experiment with `N` to get a runtime between 0.8 and 1.2 seconds. Please include a table of your raw timing measurements with your answer. There can be fairly large variation in the time for multiple runs with the same parameters (garbage collection? the JIT?). Please report raw data for five runs and take the median as your result.

Raw timing measurements for prime finding (5 runs each)

Processes	N	Run 1 (s)	Run 2 (s)	Run 3 (s)	Run 4 (s)	Run 5 (s)	Median (s)
1	4	1.010951936	0.93631317	0.598891194	0.59173145	1.127399569	0.93631317
4	18	0.929759773	1.066448848	1.29105436	0.928523512	0.929615669	0.929759773
16	48	1.228194118	0.893906001	1.089954106	0.880925032	0.880491354	0.893906001
32	40	0.811876568	0.90504187	0.911346981	0.892690169	1.123916501	0.90504187
64	5	0.778705963	1.110071623	0.942384648	1.025461248	0.765391669	0.942384648
128	5	0.80909066	0.833694351	0.724388188	0.751066255	1.145443598	0.80909066

(d) **(2 points)** If we take speed-up to mean the ratio of *throughput*, i.e.

$$\frac{\textit{PrimesFoundPerSecond} \text{ with } \texttt{NW} \text{ processes}}{\textit{PrimesFoundPerSecond} \text{ with one process}}$$

What choice for `NW` (from 4, 16, 32, 64, 128, 256) gets the highest speed-up? What is this peak speed-up? Feel free to try other values for `NW` if you want, but this isn't required to get full credit. You do need to report results for the six values for `NW` given above to get full credit.

Speed-up analysis for parallel prime finding

Processes	N	Median Time (s)	Primes/Second	Speed-up
1	4	0.93631317	4.272	1.000
4	18	0.929759773	19.360	4.532
16	48	0.893906001	53.697	12.569
32	40	0.90504187	44.197	10.346
64	5	0.942384648	5.306	1.242
128	5	0.80909066	6.180	1.447

The `NW` that gets the highest speed-up is **16 processes** with a peak speed-up of **12.569**.

2 Supporting Materials

Section 2.1 gives a brief summary of the motivation for each question on this assignment.

Section 2.2 sketches the number-theory used in the Rabin-Miller algorithm.

Section 2.2.2 describes coding guidelines for CPSC 418.

2.1 Why?

Question 0:

The course collaboration and plagiarism policies were stated in the Jan. 6 lecture: collaboration is

good; plagiarism is bad. This Question makes collaboration explicit, and plagiarism remains seriously bad.

Question 1:
This question was inspired by questions asked after lecture on January 8. The reason is to get you some practice on "eye-brain" coordination so that patterns become easy to read if you haven't programmed in language with patterns before. This should help you see how you can write your functions with patterns rather than lots of `if` expressions.

I thought of putting function templates in `hw1.erl` but decided against it because you could fill-in the code so the Erlang compiler and run-time would do all the matching and you wouldn't have to think about it. Furthermore, the Erlang compiler generates a warning if there is some pattern for which anything that would match that pattern would match an earlier pattern. There isn't a way to write a function that has a clause for each pattern from Questions 1 what doesn't trigger such a warning.

Even though I didn't provide you with function templates for this question, I *strongly* encourage you to write your own functions that use these patterns. Then try calling them with the different `Values` from the table, or try your own values. This should help you bettern understand how pattern matching works.

Question 2: `mpow`
Russian-Peasant is a cool way to exponentiate – so, you should see it. Also, the Rabin-Miller primality test would be very slow if we didn't have a fast algorithm for exponentiation. Needs a bit more thought to write the code than the brute-force method.

Note: I had considered asking you to measure the times for `mpow(X, N, M)` when `M` is larger than a single machine word (e.g., needs 1024 to 2048 bits for RSA-2048 keys). I've run some more measurements. Erlang appears to use a brute-force $\mathcal{O}(log(N)\log(M))$ algorithm for computing the product of M time N.

Question 3: Rabin-Miller
Another cool algorithm. Brings up more implementation design issues; so, it's a natural next step for bringing everyone up-to-speed with programming in Erlang. BTW, if you don't like my "functional" presentation of the algorithm, look up the Wikipedia entry (or other on-line sources) and you'll find pseudo-code with for-loops. Of course, make sure you cite your sources when you submit your solution.

Question 4: `find_primes`
Mostly just a step to get to the parallel version in Questions 5. In other words, if your parallel version doesn't call `find_primes`, you probably did too much work.

Question 5: `par_primes`
This is a course on parallel programming. So, here's something you can do in parallel.

2.2 Details on Rabin-Miller

My goal here is to take the "magic" out of the algorithm and keep everything simple and informal. Anyone who has taken MATH 312 or similar has probably seen all of these theorems before and may want to skip this section, especially if you're easily offended by informality. In the following, I will write $X \equiv_p Y$ to denote $(X \bmod P) = (Y \bmod P)$.

2.2.1 Fermat's Little Theorem

Not to be confused with Fermat's Last Theorem which is much harder to prove. OTOH, the "Little Theorem" wins for being useful.

Fermat's Little Theorem: If P is prime, and A is an integer that is not a multiple of P, then

$$A^{P-1} \equiv_p \quad = \quad 1$$

When reasoning about arithmetic mod P, it is sufficient just to consider values in $\{0, \ldots, (P-1)\}$. If A is such an integer, and A is not a multiple of P, that just means that A is in $\{1, \ldots, (P-1)\}$. Let S_P be the set $\{1, \ldots, (P-1)\}$.

1. If $A \in S_P$, then for any integer $i \geq 0$, then $A^i \in S_P$. This is because P is prime, and the product of any two positive integers that are less than P cannot be a multiple of P.

2. If $A \in S_P$, then all elements of the sequence $A^0 \bmod P$, $A^1 \bmod P$, $A^2 \bmod P, \ldots$ must be elements of S_P. By the pigeon-hole principle, there must be some value, B that appears multple times in the sequence. Choose B to be the first such state (note: we could show $B = 1$, but that's slightly more work). Let C be the length of the cycle. I.e. C is the smallest integer such that $B \equiv_p B * A^C$. Thus, $B, B * A \bmod P, B * A^2 \bmod P, \ldots, B * A^C \bmod P$ are a cycle.

3. Now we show that C must be a factor of $P-1$. If $C = P-1$, then we're done. Otherwise, choose some element of S_P that is not in the cycle, call it D. The sequence $D * A^0 \bmod P, D * A^1 \bmod P, \ldots D * A^C \bmod P$ must form a new cycle where no elements on this new cycle are elements of the original cycle. We can keep doing this until every element of S_P is an element of one of these cycles; these cycles are all disjoint; and all cycles have length C. Thus, if $E \in S_P$, $E * A^C \equiv_p E$ which implies $A^C \equiv_p 1$.

4. Let F be the total number of cycles found above. Because every element of S_P is on exactly one cycle, $F * C = |S_P| = P - 1$. Therefore

$$A^{P-1} \equiv_p \left(A^C\right)^F \equiv_p 1^F = 1$$

This completes the proof.

The Fermat Test – some examples

If `Q` is an alleged prime, we'll can randomly choose some `A` in $\{2, \ldots, Q-1\}$ and check if `mpow(A, Q-1, Q) == 1`. We'll call this the *Fermat-test*. For example, `11` is prime; and `5` is an integer that is not a multiple of `11`; and the Fermat test yields:

```
mpow(5, 10, 11)  =  misc:mod(ipow(5, 10), 11)
                 =  misc:mod(9765625, 11)
                 =  misc:mod(887784*11 + 1, 11)
                 =  1.
```

Most of the time, $A^{Q-1} \not\equiv_q 1$ if Q is not prime, for example, For example, `12` is not prime; `5` is an integer that is not a multiple of `12`; and

```
mpow(5, 11, 12)  =  misc:mod(ipow(5, 11), 12)
                 =  misc:mod(48828125, 12)
                 =  misc:mod(4069010*12 + 5, 12)
                 =  5.
```

Thus, `12` fails the Fermat-test, and we have shown that `12` is not prime.

Unfortunately, there are numbers called Fermat pseudoprimes that are composite, but pass the Fermat test for at least one choice of `A`. For example, `21` is not prime, `13` in an integer that is not a multiple of `21`, and

```
mpow(13, 20, 21)  =  misc:mod(ipow(13, 20), 21)
                  =  misc:mod(190049637748807994388 01, 21)
                  =  misc:mod(904998274994323782800*21 + 1, 21)
                  =  1.
```

In fact, there are pathological choices for `Q` where `Q` is composite and `mpow(A, (Q-1), Q) = 1` for all integers `A` that are not multiples of `Q`. Such numbers are called Carmichael numbers. 561 is the smallest Carmichael number.

2.2.2 The square-root of 1 (mod P)

If P is a prime and A is an integer such that $A * A \bmod P = 1$, then either $A \bmod P = 1$ or $A \bmod P = P - 1$.

Proof. Let A be an integer such that $A * A \equiv_p 1$.

$$\begin{array}{rcll} A * A & \equiv_p & 1, & \text{given} \\ A * A - 1 & \equiv_p & 0, & \text{subtract 1 from both sides} \\ (A+1) * (A-1) & \equiv_p & 0, & \text{factoring} \end{array}$$

If the product of integers X and Y is a multiple of a prime, at least one of X or Y must be a multiple of P. Therefore, either $(A+1)$ or $(A-1)$ must be a multiple of P which proves the claim.

The Square Test – some examples

The contrapositive of the observation that we made above is that if we can find an integer A such that $A \not\equiv_q 1$, $A \not\equiv_q Q - 1$, and $A * A \equiv_q = 1$, then Q is not prime. We call this the *square-test*.

As described in the Question 3, the Rabin-Miller algorithm factors `Q-1 = D*ipow(2, S)` and computes `mpow(A, Q-1, Q)` by first computing `mpow(A, D, Q)` and squaring the result `S` times. If they observe a value, `X`, such that `X`$\not\equiv_q 1$, `X`$\not\equiv_q$(`Q` $- 1$), and `X*X` $\equiv_q 1$, then by the square-test, `Q` is not a prime. OTOH, if after squaring `S` times, they don't reach `1`, then by the Fermat-test, `Q` is not prime. The proof that this fails with a probability of at most $\frac{1}{4}$ and that trials are independent for independently chosen values of `A` is beyond the scope of this description.

Going back to our previous example of showing that `21` passes the Fermat test when `A=13`. For the Rabin-Miller test:

```
          Q  =  21
        Q-1  =  20
          D  =  5
          S  =  2
          A  =  13
mpow(13, 5, 21)  =  13
misc:mod(13*13, 21)  =  1
```

The square-test fails, and Rabin-Miller refutes the primality of `21`.

The Library, Errors, Guards, and other good stuff

The CPSC 418 Erlang Library: your code *must* run on the CS department linux machines.

To access the course library from the CS department machines, give the following command in the Erlang shell:

```
1> code:add_path("/home/c/cs-418/public_html/resources/erl").
```

You can also set the path from the command line when you start Erlang. I've included the following in my `.bashrc` so that I don't have to set the code path manually each time I start Erlang:

```
function erl {
  /usr/bin/erl erl -eval 'code:add_path("/home/c/cs-418/public_html/resources/erl")' "$@" }
```

See http://erlang.org/doc/man/erl.html for a more detailed description of the `erl` command and the options it takes.

If you are running Erlang on your own computer, you can get a copy of the course library from http://www.students.cs.ubc.ca/~cs-418/resources/erl/erl.tgz
Unpack it in a directory of your choice, and use code:add_path as described above to use it. Changes may be made to the library to add features or fix bugs as the term progresses. I try to minimize the disruption and will announce any such changes.

Compiler Errors: if your code doesn't compile, it is likely that you will get a zero on all coding questions. Please do not submit code that does not compile successfully. After grading all assignments that compile successfully, we *might* look at some of the ones that don't. This is entirely up to the discretion of the instructor. If you have half-written code that doesn't compile, please comment it out or delete it.

Compiler Warnings: your code should compile without warnings. In my experience, most of the Erlang compiler warnings point to real problems. For example, if the compiler complains about an unused variable, that often means I made a typo later in the function and referred to the wrong variable, and ended up not using the one I wanted. Of course, the "base case" in recursive function often has unused parameters – use a `_` to mark these as unused. Other warnings such as functions that are defined but not used, the wrong number of arguments to an io:format call, etc., generally point to real mistakes in the code. We will take off points for compiler warnings.

Printing to stdout: please don't unless we specifically ask you to do so. If you include a *short* error message when throwing an error, that's fine, but not required. If you print *anything* for a case with normal execution when no printing was specified, we will take off points.

Guards: in general, guards are a good idea. If you use guards, then your code will tend to fail close to the actual error, and that makes debugging easier. Guards also make your intentions and assumptions part of the code. Documenting your assumptions in this way makes it much easier if someone else needs to work with your code, or if you need to work with your code a few months or a few years after you originally wrote it. There are some cases where adding guards would cause the code to run much slower. In those cases, it can be reasonable to use comments instead of guards. Here are a few rules for adding guards:

- If you need the guard to write easy-to-read patterns, use the guard. For example, to have separate cases for `N > 0` and `N < 0`.
- If adding the guard makes your code easier to read (and doesn't have a significant run-time penalty), use the guard.
- If a function is an "entry point" into your code (e.g. an exported function) it's good to have your assumptions about arguments clearly stated. Ideally, you this with guards, that is great.
  - Often, a function can only be implemented for *some* values of its arguments. For example, we might have:

    ```
    sendSquare(Pid, N) -> Pid !  N*N.
    ```

    A call such as `SendSquare([1, 2, 3], cow)` doesn't make sense. Bad calls should throw an error (e.g. a `badarg` error). Please, don't silently ignore bad arguments, for example

    ```
    sendSquare(Pid, N) ->
      if is_pid(Pid) and is_number(N) -> Pid ! {square, N*N};
         true -> "messed up actual arguments" % Don't do this.
      end
    end.
    ```

    The caller might very well ignore the return value of `SendSquare`, and `Pid` might end up blocking, waiting for a message that will never arrive. Furthermore, putting tests to see if the return value is an error code is so C, but throwing explicit exceptions and writing error handlers (if you want to do something other than killing the process that threw the error) is so much easier to write, read, and maintain.

We will test your code on bad arguments and make sure that an error gets thrown.

- Adding lots of little guards to every helper function can clutter your code. Write the code that you would want others to write if you are going to read it.
- In some cases, guards can cause a severe performance penalty. In that case, it's better to use a wrapper function so you can test the guards once and then go on from there. Or you can use comments; comments don't slow down the code. Any exported function should throw an error when called with bad arguments.

A common case for omitting guards occurs with tail-recursive functions. We often write a wrapper function that initializes the "accumulator" and then calls the tail-recursive code. We export the wrapper, but the tail-recursive part is not exported because the user doesn't need to know the details of the tail-recursive implementation. In this case, it makes sense to declare the guards for the wrapper function. If those guarantee the guards for the tail-recursive code, and the tail recursive code can only be called from inside its module, then we can omit the guards for the tail-recursive version. This way, the guards get checked once, but hold for all of the recursive calls. Doing this gives us the robustness of guard checking **and** the speed of tail recursion.